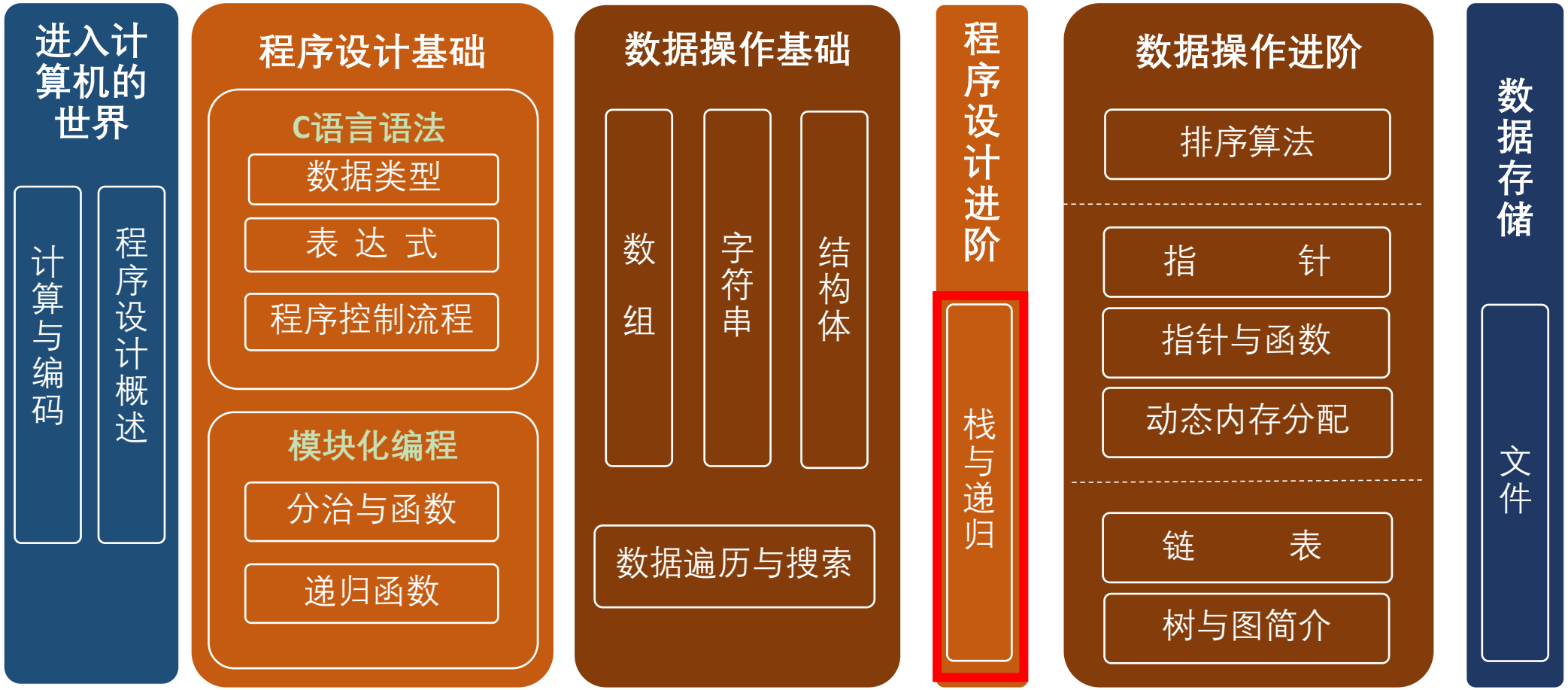
# 第十四讲 《栈与递归 2》

哈尔滨工业大学（深圳）

本课程课件是在哈尔滨工业大学苏小红老师《高级语言程序设计》课件基础上，由哈工大（深圳）《程序设计思维与实践》课程组编写制作

# 《程序设计思维与实践》课程内容结构




程序设计思想与数据操作方法融会贯通，内容由浅入深

# 学习内容

## 栈的应用举例

1 数制转换

2 括号匹配的检验

3 行编辑程序问题

4 表达式求值

## 栈与递归的实现

# 14.1 数制转换

- **数制转换**----是计算机实现计算的基本问题。

例 将10进制1348转换成8进制

✓方法：除留余数法。 $(1348)_{10}=(2504)_{8}$

计算顺序 ↓

| N | N / 8 | N % 8 |
|---|---|---|
| 1348 | 168 | 4 |
| 168 | 21 | 0 |
| 21 | 2 | 5 |
| 2 | 0 | 2 |

↑ 输出顺序

输出的时候是从高位到低位进行，恰好和计算相反。
转换的过程满足栈的后进先出的特点

# 14.1 数制转换

- 10进制数N转换成8进制的算法

```
void conversion () {
    InitStack(S);
    scanf (“%d”,N);//输入一个任意非负十进制整数
    while (N) {
      Push(S, N % 8); N = N/8;        //余数入栈
    }
    while (!StackEmpty(S)) {
      Pop(S,e);//若栈不空，则删除S的栈顶元素，用e返回其值
      printf ( "%d", e );
    }
} // conversion
```

# 14.1 数制转换

- **10进制数N转换成8进制的C语言实现代码**

```c
// 十进制转换为八进制
void DecimalToOctal(int decimalNumber) {
    SqStack S;
    InitStack(&S, 100); // 假设栈最大容量为 100

    // 处理十进制转八进制的核心逻辑
    while (decimalNumber != 0) {
        Push(&S, decimalNumber % 8);
        decimalNumber /= 8;
    }

    printf("对应的八进制数为: ");
```

```c
    // 从栈中逐个弹出并打印结果
    DataType x;
    while (!IsEmpty(&S)) {
        Pop(&S, &x);
        printf("%d", x);
    }
    printf("\n");

}
```

# 14.2 括号匹配的检验

**问题定义：** 一个表达式中，包含三种括号“(”和“)”，“[”和“]”和“{”和“}”，这三种括号可以按任意的合法次序使用。设计算法检验表达式中所使用括号的合法性。

例在表达式中正确的格式：

（［］（））或［（［］［］）］

例在表达式中不正确的格式

- ［（］）----左右不匹配
- （［（）]---左括号多了
- [（）］)----右括号多了

# 14.2 括号匹配的检验: 算法过程

[ ( [ ] [ ] ) ]

1）凡出现左括弧，则进栈；

2）凡出现右括弧，

首先检查栈是否空，若栈空，则表明该“右括弧”多余，

否则和栈顶元素比较，

若相匹配（是对应类型的左括弧），则“左括弧出栈”，

否则表明不匹配。

3）表达式检验结束时，

若栈空，则表明表达式中匹配正确，

否则表明“左括弧”有余。

# 14.2 括号匹配的检验算法描述(伪代码)

```
Status  check( ) {
   char ch;  InitStack(S);
   while ((ch=getchar())!=‘#') {
      switch (ch) {      //伪代码，理解语义，语法不对
         case (‘(’ || ‘[’ || ‘{’): Push(S,ch); break;
         case ')':
            if (StackEmpty(S)) return FALSE;
            else
               {Pop(S,e);if(e!= '(') return FALSE;}
            break;
```

# 14.2 括号匹配的检验：算法描述（伪代码）

```
 case ‘]':
   if (StackEmpty(S)) retrun FALSE;
   else
   {Pop(S,e);if(e!= ‘[') return FALSE;}
    break;…….
  default: break;
}
}
if (StackEmpty(S))  return TRUE;
else return FALSE;
}
```

# 14.3 行编辑程序问题

- **问题描述：** 在用户输入一行字符的过程中，允许用户输入出差错，并在发现有误时可以及时更正。

- **解决方法**

设立一个输入缓冲区，用以接受用户输入的一行字符，然后逐行存入用户数据区，并假设“#”为退格符（当用户发现刚刚键入的一个字符是错的，可补进一个退格符，以表示前一个字符无效），“@”为退行符（当用户发现当前键入的行内差错较多或难以补救时）。

# 14.3 行编辑程序问题

例　假设从终端接受了这样两行字符：

```
whli##ilr#e（s#*s)
  outcha@putchar(*s=#++);
```

则实际有效的是下列两行：

```
while (*s)
  putchar(*s++);
```

# 14.3 行编辑程序问题实现算法

```
void LineEdit(){
  //利用字符栈S，从终端接收一行并传送至调
  //用过程的数据区
    InitStack(S);
    ch=getchar();
    while (ch != EOF) { //EOF为全文结束符
     while (ch != '\n') {……}
     …………… }
    DestroyStack(S);
}
```

# 14.3 行编辑程序问题

- 行编辑问题算法

假设从终端接受了这样两行字符：

whli##ilr#e（s#*s)

outcha@putchar(*s=#++);

```
switch (ch) {
   case '#' : Pop(S, c); break;
   case '@': ClearStack(S); break;// 重置S为空栈
   default : Push(S, ch);  break;
  }
  ch = getchar();          // 从终端接收下一个字符
```

# 14.3 行编辑程序问题

- 行编辑问题算法

```
void LineEdit(){    //利用字符栈S，从终端接收一行并传送至调用过程的数据区
    InitStack(S);
    ch=getchar();
    while (ch != EOF) { //EOF为全文结束符
        while (ch != '\n') {……}
        将从栈底到栈顶的栈内字符传送至调用过程的数据区；
        ClearStack(S);          // 重置S为空栈
        ch = getchar();
    }
    DestroyStack(S);
}
```

# 14.4 表达式求值

例　求表达式3*(2+3*5)+6的值

四则运算规则：

（1）先乘除，后加减；

（2）从左算到右；

（3）先括号内，后括号外。

由三个部分组成：操作数、运算符、界限符（界限符是必不可少的，　反映了计算的先后顺序）

不使用界限符可以无歧义的表达运算顺序么？

表达式：
- 前缀表达式（波兰式）
- 中缀表达式
- 后缀表达式（逆波兰式）

# 14.4 表达式求值

| 规则运算符在两个操作数中间 | 规则：运算符在两个操作数后面 | 规则：运算符在两个操作数前面 |
| --- | --- | --- |
| 中缀表达式 | 后缀表达式 | 前缀表达式 |
| a+b | ab+ | +ab |
| a+b-c | a b+c- | -+abc |
| a+b-c* d | ab+cd* - | -+ab* cd |

# 14.4 表达式求值

高级语言中，采用类似自然语言的中缀表达式，但计算机对中缀表达式的处理是很困难的，而对后缀或前缀表达式则显得非常简单。

后缀表达式的特点：

① 在后缀表达式中，变量（操作数）出现的顺序与中缀表达式顺序相同。

② 后缀表达式中不需要括弧（界限符）定义计算顺序，而由运算（操作符）的位置来确定运算顺序。

# 14.4 表达式求值

中缀转后缀的手算方法：

1、确定中缀表达式中各个运算符的运算顺序

2、选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数

3、如果还有运算符没被处理，就继续

$$((15 \underset{③}{\div} (7 - \underset{②}{(} 1 \underset{①}{+} 1))) \underset{④}{\times} 3) \underset{⑦}{-} (2 \underset{⑥}{+} (1 \underset{⑤}{+} 1))$$

中缀表达式

$$15\ 7\ 1\ 1\ \overset{①}{+}\ \overset{②}{-}\ \overset{③}{\div}\ 3\ \overset{④}{\times}\ 2\ 1\ 1\ \overset{⑤}{+}\ \overset{⑥}{+}\ \overset{⑦}{-}$$

后缀表达式

# 14.4 表达式求值

中缀转后缀的手算方法：

1、确定中缀表达式中各个运算符的运算顺序

运算顺序不唯一，因此对应的 后缀表达式也不唯一

2、选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数

3、如果还有运算符没被处理，就继续

A+ B * (C - D) - E / F

A+ B * (C - D) - E / F

AB CD - * + EF / -

A B C D - * E F / - +

“左优先”：只要左边的运算符能先计算，就优先算左边的

可保证运算顺序唯一；

后缀表达式：运算符在式中出现的顺序恰好为表达式的运算顺

前缀表达式不具有

# 14.4 表达式求值

后缀表达式的计算方法（手算）：

- 从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，合体为一个操作数 （注意两个操作数的左右顺序）

AB CD -* + E F/ -

A+ B * (C - D) - E / F

# 练习题:

15 7 1 1 + - ÷ 3 × 2 1 1 + + - =

# 练习题:

15 7 1 1 + - ÷ 3 × 2 1 1 + + - = 5

上述表达式转换成中缀： 15 ÷ （7-（1+1））* 3 – (2+(1+1))

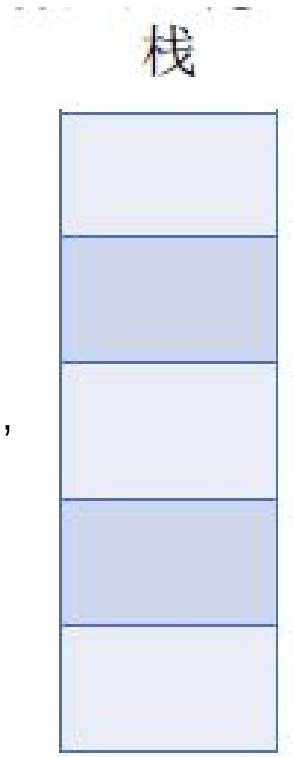
# 14.4 表达式求值

AB CD -* + E F/ -

A+ B * (C - D) - E / F

后缀表达式的计算方法（机算）：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①
- 表达式结束，栈中唯一元素即为表达式的值。



# 14.4 表达式求值（C实现，参考）

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX_SIZE 100  // 定义栈的最大容量

// 栈的定义
typedef struct {
    char data[MAX_SIZE];
    int top;
} Stack;

// 初始化栈
void initStack(Stack *stack) {
    stack->top = -1;
}

// 判断栈是否为空
int isEmpty(Stack *stack) {
    return stack->top == -1;
}
```

```c
// 判断栈是否满
int isFull(Stack *stack) {
    return stack->top == MAX_SIZE - 1;
}

// 入栈
void push(Stack *stack, char elem) {
    if (isFull(stack)) {
        printf("栈满，无法入栈！\n");
        return;
    }
    stack->data[++stack->top] = elem;
}

// 出栈
char pop(Stack *stack) {
    if (isEmpty(stack)) {
        printf("栈空，无法出栈！\n");
        return '\0';
    }
    return stack->data[stack->top--];
}
```

```c
// 获取栈顶元素
char peek(Stack *stack) {
    if (isEmpty(stack)) {
        return '\0';
    }
    return stack->data[stack->top];
}

// 判断是否为操作符
int isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/';
}

// 设置操作符优先级
int precedence(char op) {
    if (op == '+' || op == '-') {
        return 1;
    } else if (op == '*' || op == '/') {
        return 2;
    }
    return 0;
}
```

# 14.4 表达式求值（C实现，参考）

```c
// 中缀表达式转后缀表达式
void infixToPostfix(const char *infix, char *postfix) {
    Stack stack;
    initStack(&stack);

    int i = 0, j = 0;
    while (infix[i] != '\0') {
        char ch = infix[i];

        // 如果是操作数，直接添加到后缀表达式
        if (isalnum(ch)) {
            postfix[j++] = ch;
        }
        // 如果是左括号，入栈
        else if (ch == '(') {
            push(&stack, ch);
        }
        // 如果是右括号，弹出栈中的操作符直到遇到左括号
        else if (ch == ')') {
            while (!isEmpty(&stack) && peek(&stack) != '(') {
                postfix[j++] = pop(&stack);
            }
            // 弹出左括号
            if (!isEmpty(&stack) && peek(&stack) == '(') {
                pop(&stack);
            }
        }
        // 如果是操作符
        else if (isOperator(ch)) {
            while (!isEmpty(&stack) && precedence(peek(&stack)) >= precedence(ch))
                postfix[j++] = pop(&stack);
            }
            push(&stack, ch);
        }
        i++;
    }
    // 将栈中剩余的操作符弹出
    while (!isEmpty(&stack)) {
        postfix[j++] = pop(&stack);
    }

    // 添加字符串结束符
    postfix[j] = '\0';
}
```

# 14.4 表达式求值（C实现，参考）

```c
int main() {
    char infix[MAX_SIZE];
    char postfix[MAX_SIZE];

    printf("请输入中缀表达式: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("后缀表达式: %s\n", postfix);

    return 0;
}
```

# 14.5 栈与递归的实现 （递归的简要回顾）

- **递归调用的定义：**
  - 子程序（或函数）直接调用自己或通过一系列调用语句间接调用自己。是一种描述问题和解决问题的基本方法。
- **递归的基本思想：**
  - 把一个不能或不好求解的大问题转化为一个或几个小问题，再把这些小问题进一步分解成更小的小问题，直至每个小问题都可以直接求解。

$$n! = \begin{cases} 1 & \text{当} n=0 \text{或} n=1 \\ n*(n-1)! & \text{当} n>1 \end{cases}$$

- **递归的要素：**
  - 递归边界条件：确定递归到何时终止，也称为递归出口；
  - 递归模式：大问题是如何分解为小问题的，也称为递归体

# 14.5 栈与递归的实现

- 什么问题可以应用递归
  - 一类是递归函数
  - 第二类是，有的数据结构，如二叉树，广义表等，由于结构本身固有的递归特性，则它们的操作可递归地描述
  - 还有一类问题，虽然问题本身没有明显的递归结构，但用递归求解比迭代求解更简单，如八皇后问题、Hanoi塔问题等

# 14.5 栈与递归的实现

- **函数调用：**
  - 当在一个函数的运行期间调用另一个函数时，在运行该被调用函数之前，需先完成三件事：
    - 将所有的实在参数、返回地址等信息传递给被调用函数保存；
    - 为被调用函数的局部变量分配存储区；
    - 将控制转移到被调函数的入口。

# 14.5 栈与递归的实现

- **函数调用的背后过程：**

```
void main() {
  int a, b, c;
  ...
  func1 (a, b);
#1 c=a+b;
  ...
}
```

```
void func1 (int a, int b) {
  int x;
  ...
  func2 (x);
#2 x=x+10086;
  ...
}
```

```
void func2 (int x) {
  int m, n;
  ...
}
```

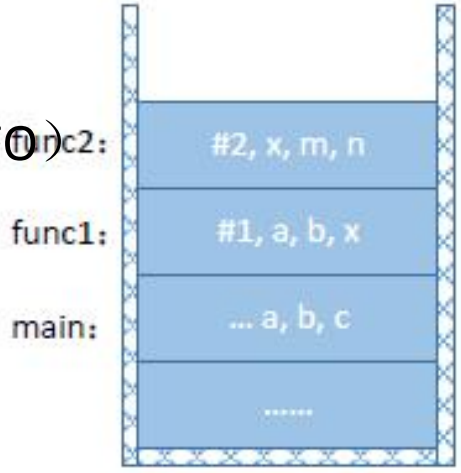
函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数调用时，需要用一个栈存储：

① 调用返回地址

② 实参

③ 局部变量




函数调用栈

# 14.5 栈与递归的实现

- 函数调用
  - 从被调用函数返回调用函数之前，应该完成：
    - 保持被调用函数的计算结果；
    - 释放被调用函数的数据区；
    - 依照被调用函数保存的返回地址将控制转移到调用函数。

# 14.5 栈与递归的实现

- **函数调用的背后过程：**

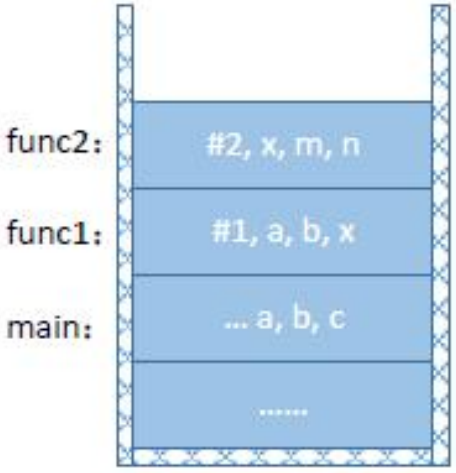
```
void main() {
  int a, b, c;
  ...
  func1 (a, b);
#1 c=a+b;
  ...
}
```

```
void func1 (int a, int b) {
  int x;
  ...
  func2 (x);
#2 x=x+10086;
  ...
}
```

```
void func2 (int x) {
  int m, n;
  ...
}
```

函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数返回时：
① 返回函数计算结果（若有）
② 释放被调用函数的数据区（弹出相应函数信息）
③ 根据保存地址转移到调用函数



函数调用栈

# 14.5 栈与递归的实现

- **函数调用：**
  - 多个函数嵌套调用的规则是：
    - 后调用先返回；
    - 此时的内存管理实行“栈式管理”

# 14.5 栈与递归的实现

【例】求阶乘的函数

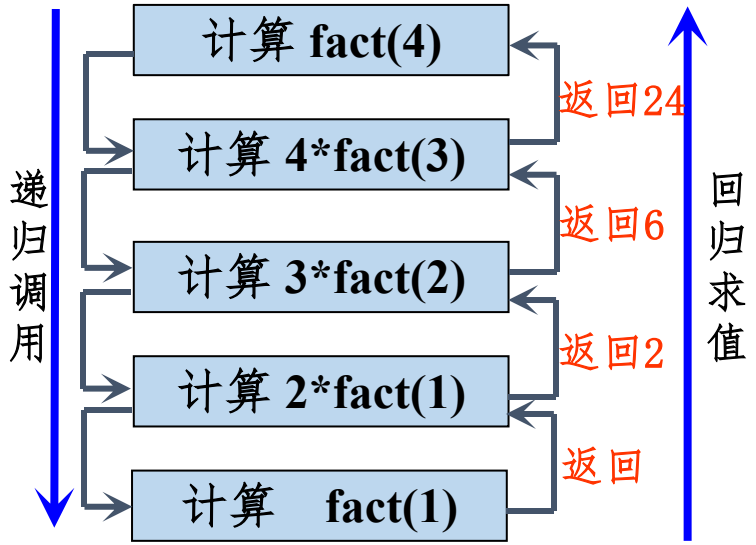
$$n! = \begin{cases} 1 & \text{当} n=0 \text{或} n=1 \\ n*(n-1)! & \text{当} n>1 \end{cases}$$

**(1) 非递归算法**

```
long fact( int n )
{
    long f=2;    int i;
    for( i=2; i <= n; i++ )
        f = f*i;
    return(f);
}
```

**(2) 递归算法**

```
long fact( int n )
{
    if ((n == 0) || (n==1))
        return 1;
    else
        return  n * fact (n-1);
}
```



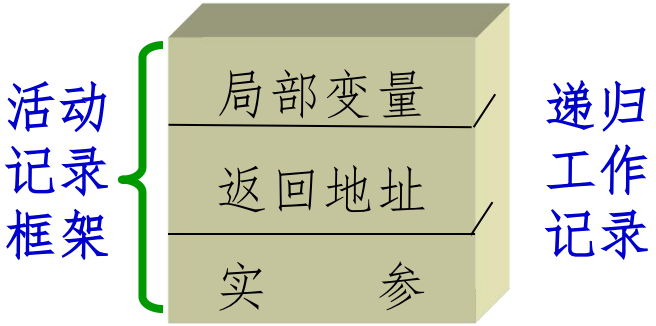

T(n)=o(n)

T(n)= ?

# 14.5 栈与递归的实现

## 递归过程与递归工作记录

①每一次递归调用时，需要为过程中使用的参数、局部变量和返回地址等另外分配存储空间；
②每层递归调用需分配的空间形成递归工作记录，按栈结构组织，即LIFO。




## 递归函数的内部执行过程

①建栈：运行开始时，首先为递归调用建立一个工作栈，其结构包括值参、局部变量和返回地址；
②压栈：每次执行递归调用之前，把递归函数的值参和局部变量的当前值以及调用后的返回地址压栈；
③出栈：每次递归调用结束后，将栈顶元素出栈，使相应的值参和局部变量恢复为调用前的值，然后转向返回地址指定的位置继续执行。

# 栈的练习：

一个栈的入栈序列是a, b, c, d, e，则栈的不可能的出栈序列是（可以按照上述先后顺序在任意时刻执行入栈和出栈操作，但每个元素只能入栈和出栈一次）？

A edcba

B cdeba

C debca

D abcde

# 栈的练习

一个栈的入栈序列是a, b, c, d, e，则栈的不可能的出栈序列是（可以按照上述先后顺序在任意时刻执行入栈和出栈操作，但每个元素只能入栈和出栈一次）？

A edcba

B cdeba

C debca

D abcde